

Introduction aux transformateurs

Richard E. Turner
Département d'ingénierie, Université de Cambridge, Royaume-Uni Microsoft
Research, Cambridge, Royaume-Uni
ret26@cam.ac.uk

Résumé. Le transformateur est un composant de réseau neuronal qui peut être utilisé pour apprendre des représentations utiles de séquences ou d'ensembles de points de données [Vaswani et al., 2017]. Le transformateur a été à l'origine des récentes avancées dans le traitement du langage naturel [Devlin et al., 2019], la vision par ordinateur [Dosovitskiy et al., 2021] et la modélisation spatio-temporelle [Bi et al., 2022]. Il existe de nombreuses introductions aux transformateurs, mais la plupart ne contiennent pas de descriptions mathématiques précises de l'architecture et les intuitions qui sous-tendent les choix de conception font souvent défaut.¹ De plus, la recherche suivant un chemin sinueux, les explications relatives aux composants du transformateur peuvent être idiosyncrasiques. Dans cette note, nous visons à fournir une description mathématiquement précise, intuitive et claire de l'architecture du transformateur. Nous n'aborderons pas la question de l'entraînement, car celle-ci est plutôt standard. Nous partons du principe que le lecteur est familiarisé avec les thèmes fondamentaux de l'apprentissage automatique, notamment les perceptrons multicouches, les transformations linéaires, les fonctions softmax et les probabilités de base.

¹ Voir Phuong et Hutter [2022] pour une exception à cette règle.

1 Préambule

Commençons par parler de la forme des données qui sont entrées dans un transformateur, de l'objectif du transformateur et de la forme de sa sortie.

1.1 Format des données d'entrée : ensembles ou séquences de jetons

Afin d'appliquer un transformateur, les données doivent être converties en un ensemble ou une séquence¹ de N jetons $x^{(0)}$ de dimension D (voir figure 1). Les jetons peuvent être rassemblés dans une matrice $X^{(0)}$ qui est $D \times N$.² Pour donner deux exemples concrets

1. un passage de texte peut être divisé en une séquence de mots ou de sous-mots, chaque mot étant représenté par un vecteur unique,
2. Une image peut être divisée en un ensemble de fragments, et chaque fragment peut être mappé dans un vecteur.

Les intégrations peuvent être fixes ou apprises avec le reste des paramètres du modèle. Par exemple, les vecteurs représentant les mots peuvent être optimisés ou une transformation linéaire apprise peut être utilisée pour intégrer les fragments d'image (voir figure 2).

Une séquence de jetons est une représentation générique à utiliser comme entrée : de nombreux types de données peuvent être « tokenisés » et les transformateurs sont alors immédiatement applicables, plutôt que de nécessiter des architectures sur mesure pour chaque modalité comme c'était le cas auparavant (CNN pour les images, RNN pour les séquences, deepsets pour les ensembles, etc.). De plus, cela signifie que vous n'avez pas besoin d'architectures sur mesure pour mélanger des données de différentes modalités : vous pouvez simplement les regrouper dans un grand ensemble de jetons.

1.2 Objectif : représentations de séquences

Le transformateur ingère les données d'entrée $X^{(0)}$ et renvoie une représentation de la séquence sous la forme d'une autre matrice $X^{(M)}$ qui est également de taille $D \times N$.

La tranche $x_n = X^{(M)}_{:,n}$ sera un vecteur de caractéristiques représentant la séquence à l'emplacement du jeton n . Ces représentations peuvent être utilisées pour la prédiction autorégressive du jeton suivant ($n+1$), la classification globale de la séquence entière (en regroupant l'ensemble de la représentation), les problèmes de prédiction séquence-à-séquence ou image-à-image, etc. Ici, M désigne le nombre de couches dans le transformateur.

2 Le bloc transformateur

La représentation de la séquence d'entrée sera produite en appliquant de manière itérative un bloc transformateur

$$X^{(m)} = \text{transformateur-bloc}(X^{(m-1)}).$$

Le bloc lui-même comprend deux étapes : l'une opérant sur la séquence et l'autre opérant sur les caractéristiques. La première étape affine chaque caractéristique indépendamment en fonction des relations entre les tokens dans la séquence, par exemple dans quelle mesure un mot dans une séquence à la position n dépend des mots précédents à la position n' , ou dans quelle mesure deux fragments différents d'une image sont liés l'un à l'autre. Cette étape agit horizontalement sur les lignes de $X^{(m-1)}$. La deuxième étape affine les caractéristiques représentant chaque token. Cette étape agit verticalement sur une colonne de $X^{(m-1)}$. En appliquant de manière répétée le bloc transformateur, la représentation au niveau du token n et de la caractéristique d peut être façonnée par les informations au niveau du token n' et de la caractéristique d' .³

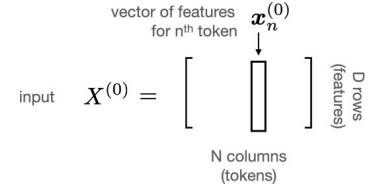


Figure 1 : L'entrée d'un transformateur est constituée de N vecteurs x_n qui sont chacun de dimension D . Ceux-ci peuvent être regroupés dans un tableau $X^{(0)}$.

¹ À proprement parler, la collection de jetons n'a pas besoin d'avoir un ordre et le transformateur peut les traiter comme un ensemble (où l'ordre n'a pas d'importance), plutôt que comme une séquence. Voir la section 3.

² Notez que la plupart des publications utilisent la notation transposée, dans laquelle la matrice de données est $N \times D$, mais je souhaite que les séquences s'étendent sur toute la page et que les caractéristiques s'alignent verticalement dans les schémas (une convention que j'utilise dans d'autres notes de cours).

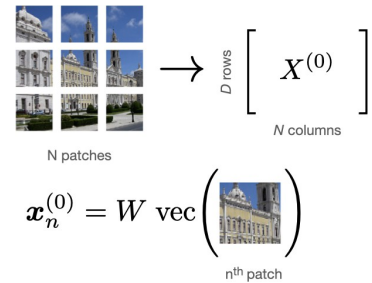


Figure 2 : Encodage d'une image : un exemple [Doso-vitskiy et al., 2021]. Une image est divisée en N patches. Chaque patch est remodelé en un vecteur par l'opérateur vec . Ce vecteur est traité par une matrice W qui mappe le patch à une dimension D .

Ces vecteurs sont collectés ensemble dans l'entrée $X^{(0)}$. La matrice W peut être apprise avec le reste des paramètres du transformateur.

³ L'idée d'entrelacer le traitement à travers la séquence et à travers les caractéristiques est un motif commun à de nombreuses architectures d'apprentissage automatique, notamment les réseaux neuronaux graphiques (entrelacement du traitement à travers les nœuds et à travers les caractéristiques), les opérateurs neuronaux de Fourier (entrelacement du traitement à travers l'espace et à travers les caractéristiques) et les blocs goulots d'étranglement dans les ResNets (entrelacement du traitement à travers les pixels et à travers les caractéristiques).

2.1 Étape 1 : auto-attention à travers la séquence

La sortie de la première étape du bloc transformateur est un autre tableau $D \times N$, $Y^{(m)}$. La sortie est produite en agrégeant les informations de la séquence indépendamment pour chaque caractéristique à l'aide d'une opération appelée *attention*.

Attention. Plus précisément, le vecteur de sortie à l'emplacement n , noté $y_n^{(m)}$, est produit par une simple moyenne pondérée des caractéristiques d'entrée à l'emplacement n' :

$1 \dots N$, noté $x^{(m-1)}$, c'est-à-dire⁴

$$y_n^{(m)} = \sum_{n'=1}^N x_{n'}^{(m-1)} A_{n',n}^{(m)} \quad (1)$$

Ici, la pondération est donnée par une *matrice* dite *d'attention* $A^{(m)}$ n', n qui est de

taille⁵ $N \times N$ et normalise sur ses colonnes $\sum_{n'=1}^N A_{n',n}^{(m)} = 1$,
Intuitivement

parler $A_{n',n}^{(m)}$ prendra une valeur élevée pour les emplacements dans la séquence n' qui sont

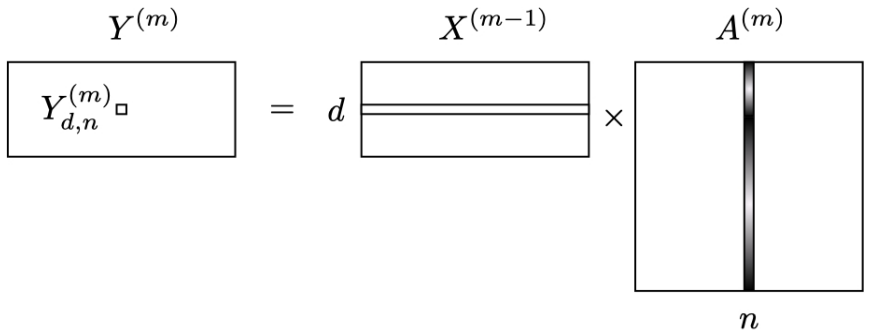
de haute pertinence pour l'emplacement n . Pour les emplacements non pertinents, il prendra la valeur

0. Par exemple, tous les patches d'une scène visuelle provenant d'un seul objet peuvent avoir des valeurs d'attention correspondantes élevées.

Nous pouvons écrire de manière compacte la relation sous forme de multiplication matricielle,

$$Y^{(m)} = X^{(m-1)} A^{(m)}, \quad (2)$$

et nous l'illustrons ci-dessous dans la figure 3.⁶



produit scalaire de l'entrée découpée horizontalement dans le temps $X^{(m)}$ $N \times N$ $d, :$

avec une tranche verticale de la matrice d'attention $A^{(m)}$. Ici, les nuances dans la matrice d'attention représentent les éléments ayant une valeur élevée en blanc et ceux ayant une valeur faible, proche de 0, en noir.

Auto-attention. Jusqu'ici, tout est simple. Mais d'où vient la matrice d'attention ? L'idée ingénieuse de la première étape du transformateur est que la matrice d'attention est générée à partir de la séquence d'entrée elle-même, ce qu'on appelle *l'auto-attention*.

Une manière simple de générer la matrice d'attention à partir de l'entrée serait de mesurer la similarité entre deux emplacements à l'aide du produit scalaire entre les caractéristiques de ces deux emplacements, puis d'utiliser une fonction softmax pour gérer la normalisation, c'est-à-dire⁷

$$A_{n,n'} = \frac{\exp(x_n^T x_{n'})}{\sum_{n''=1}^N \exp(x_n^T x_{n''})}$$

⁴ **Relation avec les réseaux neuronaux convolutifs (CNN).** Le mécanisme d'attention peut récupérer le filtrage convolutif comme cas particulier par exemple si $x^{(0)}$ est une série chronologique 1D échantillonnée régulièrement

et $A_{n',n}^{(m)} = \frac{1}{n'-n+1}$ alors le mécanisme d'attention dans l'équation 1 devient une convolution. Contrairement aux CNN normaux, ces filtres ont un support temporel complet. Nous verrons plus tard que les filtres eux-mêmes dépendent de manière dynamique de l'entrée, ce qui constitue une autre différence par rapport aux CNN standard. Nous verrons également qu'il y a une similarité : les transformateurs utiliseront plusieurs attentions

Les cartes d'attention dans chaque couche de la même manière que les CNN utilisent plusieurs filtres (bien que les transformateurs aient généralement moins de cartes d'attention que les CNN ont de canaux).

⁵ La nécessité pour les transformateurs de stocker et de calculer $N \times N$ tableaux d'attention peut constituer un goulot d'étranglement informatique majeur, ce qui rend difficile le traitement de longues séquences.

⁶ Lors de l'entraînement des transformateurs à effectuer des prédictions autorégressives d'auto- , par exemple prédire le mot suivant dans une séquence en fonction des mots précédents, une modification intelligente du modèle peut être utilisée pour accélérer l'entraînement et l'inférence. Cela implique d'appliquer le transformateur à l'ensemble de la séquence et d'utiliser le masquage dans le mécanisme d'attention ($A^{(m)}$ devient une matrice triangulaire supérieure) pour empêcher les futurs tokens d'affecter la représentation des tokens précédents. Des prédictions causales peuvent alors être effectuées pour l'ensemble de la séquence en un seul passage vers l'avant à travers le transformateur. Voir la section 4 pour plus d'informations.

⁷ Nous supprimons temporairement les exposants ici. pour simplifier la notation, donc $x_n^{(m)}$ devient x_n et de même $x_{n'}^{(m)}$ devient $x_{n'}$.

Cependant, cette approche naïve mêle les informations sur la similitude entre les emplacements dans la séquence avec le contenu de la séquence elle-même.

Une alternative consiste à effectuer la même opération sur une transformation linéaire de la séquence, Ux_n , de sorte que⁸

$$A_{n,n'} = \frac{\exp(x_n^T U^T U x_{n'})}{\sum_{n''=1}^N \exp(x_n^T U^T U x_{n''})}$$

En général, U se projette dans un espace de dimension inférieure, c'est-à-dire que U est de dimension $K \times D$ avec $K < D$. De cette manière, seules certaines des caractéristiques de la séquence d'entrée doivent être utilisées pour calculer la similarité, les autres étant projetées à l'extérieur, ce qui permet de dissocier le calcul de l'attention du contenu. Cependant, le numérateur dans cette construction est symétrique. Cela pourrait être un inconvénient. Par exemple, nous pourrions vouloir que le mot « fer à calfeutrer » soit fortement associé au mot « outil » (car il s'agit d'un type d'outil), mais que le mot « outil » soit plus faiblement associé au mot « fer à calfeutrer » (car la plupart d'entre nous le rencontrons rarement).⁹

Heureusement, il est facile de généraliser le mécanisme d'attention ci-dessus pour le rendre asymétrique en appliquant deux transformations linéaires différentes à la séquence d'origine.

$$A_{n,n'} = \frac{\exp(x_n^T U_q^T U_k x_{n'})}{\sum_{n''=1}^N \exp(x_n^T U_q^T U_k x_{n''})} \quad (3)$$

Les deux quantités qui sont ici multipliées par le produit scalaire $q_n = U_q x_n$ et $k_n =$

fonction

k_n sont généralement appelées respectivement les *requêtes* et les *clés*. Les équations 2 et 3 définissent ensemble le mécanisme d'auto-attention. Notez que les $K \times D$ matrices U_q et U_k sont les seuls paramètres de ce mécanisme.¹⁰

Auto-attention multi-têtes (MHSA). Dans les mécanismes d'auto-attention décrits ci-dessus, il existe une matrice d'attention qui décrit la similarité de deux emplacements dans la séquence. Cela peut constituer un goulot d'étranglement dans l'architecture : il serait utile que les paires de points soient similaires dans certaines « dimensions » et différentes dans d'autres.¹¹

Afin d'augmenter la capacité de la première étape d'auto-attention, le bloc transformateur applique H ensembles d'auto-attention en parallèle¹² (appelés H têtes), puis projette linéairement les résultats vers le bas vers le tableau $D \times N$ requis pour la suite du traitement. Cette légère généralisation est appelée *auto-attention multi-têtes*.

$$Y^{(m)} = \text{MHSA}_\theta(X^{(m-1)}) = \sum_{h=1}^H V^{(m)}_h X^{(m-1)} A^{(m)}_h, \quad \text{où} \quad (4)$$

$$[A^{(m)}_h]_{n,n'} = \frac{\exp(x_n^{(m)} U_{q,h}^T U_{k,h} x_{n'}^{(m)})}{\sum_{n''=1}^N \exp(x_n^{(m)} U_{q,h}^T U_{k,h} x_{n''}^{(m)})} \quad (5)$$

$$q_{h,n}^{(m)} = U_{q,h}^{(m)} x_n^{(m-1)} \quad \text{et} \quad k_{h,n}^{(m)} = U_{k,h}^{(m)} x_n^{(m-1)}. \quad (6)$$

Ici, les matrices $H V^{(m)}$ qui sont $D \times D$ projettent les étapes d'auto-attention H

jusqu'à la dimensionnalité de sortie requise D .

L'ajout des matrices $V^{(m)}$ et le fait de ne conserver que la diagonale

de la matrice d'attention $A^{(m)}$ interagira instantanément avec le signal.

avec elle-même, signifie qu'il existe un certain traitement croisé des caractéristiques dans l'auto-attention multi-têtes, par opposition à un traitement purement croisé des séquences. Cependant, l'étape a une capacité limitée pour ce type de traitement et c'est le rôle de la deuxième étape de s'en occuper.

⁸ Souvent, vous verrez l'attention paramétrée comme

$$A_{n,n'} = \frac{\exp(x_n^T U^T U x_{n'} / D)}{\sum_{n''=1}^N \exp(x_n^T U^T U x_{n''} / D)}.$$

En divisant les exposants par la racine carrée de dimensionnalité du vecteur projeté contribue à la stabilité numérique, mais dans cette présentation, nous absorbons ce terme dans U pour plus de clarté.

⁹ Une partie de cet effet pourrait être gérée par la normalisation dans le dénominateur, mais la similarité asymétrique offre plus de flexibilité. Cependant, je ne connais aucune preuve expérimentale qui soutienne l'utilisation de $U_q \neq U_k$.

¹⁰ Relation avec les réseaux neuronaux récurrents

fonctionnent (RNN). Il est instructif de comparer le traitement temporel dans le transformateur à celui

celle des RNN qui mettent à jour de manière récurrente un état caché en fonction de la représentation des caractéristiques de l'état caché ($x^{(t)}$)

sur l'observation actuelle ($x_n^{(t)}$) et la pré-état caché précédent $x_{n-1}^{(t-1)}$. Ici, nous avons déroulé le RNN d'un pas pour montrer que les observations proches de l'état caché (par exemple $x^{(0)}$) sont traitées différemment des observations plus éloignées (par exemple $x^{(n)}$), car les informations sont propagées par l'application récurrente de la fonction $f(\cdot)$. En revanche, dans le transformateur, l'auto-attention traite toutes les observations à tous les moments de manière identique, quelle que soit leur distance. C'est l'une des raisons pour lesquelles ils trouvent plus simple d'apprendre les relations à longue distance.

¹¹ Si les matrices d'attention sont considérées comme une version des filtres d'un CNN basée sur les données, alors la nécessité d'avoir plus de filtres/canaux est évidente. Typique

Les choix pour le nombre de têtes H sont 8 ou 16, ce qui est inférieur au nombre habituel de canaux dans un CNN.

¹² Le coût de calcul de l'auto-attention multi-têtes est généralement dominé par la multiplication matricielle impliquant la matrice d'attention et est donc $O(HDN^2)$.

¹³ Le produit des matrices $V^{(m)} X^{(m-1)}$ est lié aux valeurs dites *wh* qui sont nor-

mally introduites dans les descriptions de l'auto-attention aux côtés des requêtes et des clés. Dans la présentation habituelle

, il existe une redondance entre les dimensions linéaires utilisées pour calculer les valeurs et la projection linéaire à la fin de la matrice d'attention multi-têtes

attention, nous ne les avons donc pas explicitement présentés ici. La présentation standard peut être réutilisée

couvert en définissant V_h comme une matrice de rang faible $V_h = U_h U_{v,h}$ où U_h est $D \times K$ et $U_{v,h}$ est $K \times D$. Généralement, K est défini comme $K = D/H$ afin que la modification du nombre de têtes conduise à des modèles avec un nombre similaire de paramètres et d'exigences de calcul.

La figure 4 montre schématiquement l'auto-attention multi-têtes. L'attention multi-têtes comprend les paramètres suivants $\theta = \{U_{q,h}, U_{k,h}, V_h\}_{h=1}^H$ c'est-à-dire $3H$ matrices de taille $K \times D$, $K \times D$ et $D \times D$ respectivement.

Figure 4 . L'auto-attention multi-têtes applique des opérations d'auto-attention H en parallèle, puis projette linéairement la sortie HD N dimensionnelle vers D N en appliquant une transformation linéaire, mise en œuvre ici par les matrices HV_h .

La deuxième étape du traitement dans le bloc transformateur opère sur l'ensemble des caractéristiques, affinant la représentation à l'aide d'une transformation non linéaire. Pour ce faire, nous appliquons simplement un perceptron multicouche (MLP) au vecteur de caractéristiques à chaque emplacement n de la séquence,

Notez que les paramètres du MLP, θ , sont les mêmes pour chaque emplacement n .¹⁴

De manière équivalente, cela peut être considéré comme une modélisation des différences entre la représentation $x^{(m)} - x^{(m-1)} = \text{res}_\theta(x^{(m-1)})$ et fonctionnera bien lorsque la fonction modélisée est proche de l'identité. Ce type de paramétrage est utilisé à la fois pour les étapes MHSA et MLP dans le transformateur, l'idée étant que chacune applique une transformation non linéaire modérée à la représentation. Sur plusieurs couches, ces transformations non linéaires modérées se combinent pour former des transformations importantes.

où $\text{moyenne}(\mathbf{x}_n) = \frac{1}{D} \sum_{d=1}^D x_{d,n}$ et $\text{var}(\mathbf{x}_n) = \frac{1}{D} \sum_{d=1}^D (x_{d,n} - \text{mean}(\mathbf{x}_n))^2$.
Les deux paramètres γ_d et β_d sont une échelle et un décalage appris.

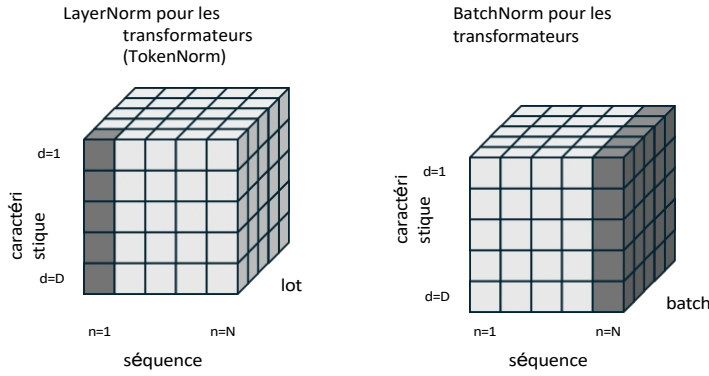


Figure 5 : Les transformateurs effectuent une normalisation par couche (schéma de gauche) qui normalise la moyenne et l'écart type de chaque token individuel dans chaque séquence du lot. La normalisation par lot (schéma de droite), qui normalise à la fois la dimension des caractéristiques et celle du lot, s'avère beaucoup moins stable [Shen et al., 2020]. D'autres types de normalisation sont possibles et po-

potentiellement sous-explorée, par exemple la normalisation d'instance normaliserait plutôt à travers la dimension de la séquence.

Comme cette transformation normalise chaque jeton individuellement et que LayerNorm est appliqué différemment dans les CNN (voir figure 6), je préfère appeler cette normalisation TokenNorm.

Cette transformation empêche les représentations de prendre une ampleur démesurée lorsque des non-linéarités sont appliquées de manière répétée à travers les réseaux neuronaux.¹⁷ Dans les transformateurs, LayerNorm est généralement appliqué dans les termes résiduels des étapes MHSA et MLP.

En rassemblant tout cela, nous obtenons le bloc transformateur standard représenté schématiquement à la figure 7.¹⁸

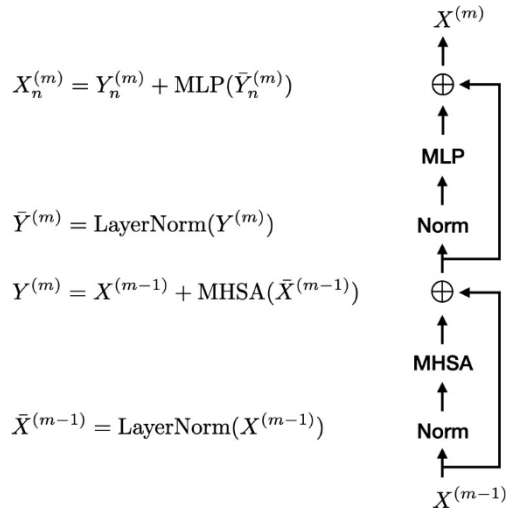


Figure 7 : Le bloc transformateur. Des connexions résiduelles sont ajoutées à l'étape d'auto-attention multi-têtes (MHSA) et à l'étape de perceptron multicouche (MLP). La normalisation des couches est également appliquée aux entrées du MHSA et du MLP. Elles sont ensuite empilées. Ce bloc peut alors être répété M fois.

3 Codage de position

Le transformateur traite les données comme un ensemble : si vous permuter les colonnes de $X^{(0)}$ (c'est-à-dire que vous réorganisez les jetons dans la séquence d'entrée), vous permuter toutes les représentations à travers le réseau $X^{(m)}$ de la même manière. Cela est essentiel pour de nombreuses applications, car il n'existe pas nécessairement de manière naturelle d'ordonner les données d'origine en une séquence de jetons. Par exemple, il n'existe pas d'ordre « correct » unique pour mapper

¹⁷ S'il est possible de contrôler les non-linéarités et les poids dans les réseaux neuronaux afin d'empêcher l'explosion de la représentation, les contraintes que cela impose aux fonctions d'activation peuvent nuire à l'apprentissage. L'approche de LayerNorm est sans doute plus simple et plus facile à former.

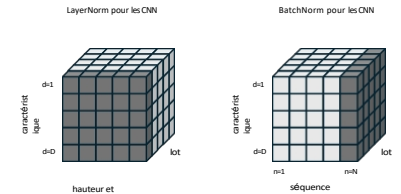


Figure 6 : Dans les CNN, LayerNorm est généralement appliqué à la fois aux caractéristiques et à l'ensemble des cartes de caractéristiques (c'est-à-dire à la hauteur et à la largeur des images) (schéma de gauche). Étant donné que les dimensions hauteur et largeur dans les CNN correspondent à la dimension séquentielle $1 \dots N$ des transformateurs, le terme « LayerNorm » est sans doute utilisé de manière incohérente (comparer avec la figure 5). Je préférerais appeler la normalisation utilisée dans les transformateurs « normalisation des tokens » afin d'éviter toute confusion. La normalisation par lots (schéma de droite) est définie de manière cohérente.

¹⁸ La configuration exacte des couches de normalisation et résiduelles peut varier, mais nous présentons ici une configuration standard [Xiong et al., 2020].

patches d'images en une séquence unidimensionnelle.

Cependant, cela pose un problème, car les informations de position sont essentielles dans de nombreux cas et le transformateur les a supprimées. La séquence « les herbivores mangent des plantes » ne devrait pas avoir la même représentation (à permutation près) que « les plantes mangent les herbivores ». De même, une image ne devrait pas avoir la même représentation qu'une image composée des mêmes fragments permutés de manière aléatoire. Heureusement, il existe une solution simple à ce problème : l'emplacement de chaque token dans l'ensemble de données d'origine doit être inclus dans le token lui-même ou dans la manière dont il est traité. Il existe plusieurs options pour y parvenir, l'une d'entre elles consistant à inclure ces informations directement dans l'intégration $X^{(0)}$. Par exemple, en ajoutant simplement l'intégration de position (étonnamment, cela fonctionne¹⁹) ou en concaténant. Les informations de position peuvent être fixées, par exemple en ajoutant un vecteur de sinusoides de différentes fréquences et phases pour coder la position d'un

mot d'une phrase [Vaswani et al., 2017], ou peut être un paramètre libre qui est appris [Devlin et al., 2019], comme cela se fait souvent dans les transformateurs d'images. Il existe également des approches visant à inclure des informations de distance relative entre des paires de tokens en modifiant le mécanisme d'auto-attention [Wu et al., 2021], qui est lié aux transformateurs équivariants.

4 Variantes de transformateurs spécifiques à une application

Pour être complet, nous donnerons quelques exemples simples illustrant comment l'architecture standard du transformateur ci-dessus est utilisée et modifiée pour des applications spécifiques. Ceci

comprend l'ajout d'une tête aux blocs du transformateur afin d'effectuer la tâche de prédiction souhaitée, mais également des modifications à la construction standard du corps.

4.1 Modélisation autorégressive du langage

Dans la modélisation autorégressive du langage, l'objectif est de prédire le mot suivant w_n dans la séquence donnée par les mots précédents $w_{1:n-1}$, c'est-à-dire de renvoyer $p(w_n = w | w_{1:n-1})$. Deux modifications sont nécessaires pour utiliser le transformateur pour cette tâche

— une modification du corps pour rendre l'architecture plus efficace et l'ajout d'une tête pour faire les prédictions du mot suivant.

Modification du corps : masquage autorégressif. L'application de la version du transformateur que nous avons abordée jusqu'à présent à la prédiction autorégressive est coûteuse en termes de calcul, tant pendant l'entraînement que pendant les tests. Pour s'en rendre compte, il suffit de noter que la prédiction AR nécessite d'effectuer une séquence de prédictions : vous commencez par prédire le premier mot $p(w_1 = w)$, puis vous prédisiez le deuxième mot à partir du premier $p(w_2 = w | w_1)$, puis le troisième mot étant donné les deux premiers $p(w_2 = w | w_1, w_2)$, et ainsi de suite jusqu'à ce que vous prédisiez le dernier élément de la séquence $p(w_N = w | w_{1:N-1})$. Cela nécessite d'appliquer le transformateur $N - 1$ fois avec des séquences d'entrée qui augmentent d'un mot à chaque fois : $w_1, w_{1:2}, \dots, w_{1:N-1}$. Cela est très coûteux tant au niveau du temps d'entraînement que du temps de test.

Heureusement, il existe une solution élégante à ce problème : permettre au transformateur de prendre en charge les mises à jour incrémentielles, grâce auxquelles, si vous ajoutez un nouveau jeton à une séquence existante, vous ne modifiez pas la représentation des anciens jetons. Pour clarifier cette propriété, je vais la définir mathématiquement : soit la sortie du transformateur incrémentiel appliqué aux n premiers mots notée²⁰

$$X^{(n)} = \text{transformateur-incrémentiel}(w_{1:n}).$$

La sortie du transformateur incrémentiel appliqué à $n + 1$ mots est alors

$$X^{(n+1)} = \text{transformateur-incrémentiel}(w_{1:n+1}).$$

Dans le transformateur incrémentiel $X^{(n)} = X^{(n+1)}_{1:D, 1:n}$, c'est-à-dire la représentation de anciens jetons n'a pas changé par l'ajout du nouveau. Si nous avons cette propriété

¹⁹ Transformateurs de vision [Dosovitskiy et al., 2021]

(0) $x_n = \frac{1}{\sqrt{p}} W p^n + e$ où p^n est le n -ième patch vectorisé, e est l'intégration de position apprise, et W est la matrice d'intégration du patch. Il serait sans doute plus intuitif d'ajouter l'intégration de position à l'intégration du patch. Cependant, si nous utilisons l'approche de concaténation et considérons ce qui se passe après l'application d'une transformation linéaire,

$$\begin{aligned} V \begin{matrix} W_{pn} \\ e \end{matrix} &= \begin{matrix} V_{11} & V_{12} \\ V_{21} & V_{22} \end{matrix} \begin{matrix} W_{pn} \\ e \end{matrix} \\ &= \begin{matrix} V_{11}W_{pn} + V_{12}e \\ V_{21}W_{pn} + V_{22}e \end{matrix} \\ &= W'_{pn} + e' \end{aligned}$$

Nous retrouvons la construction additive, ce qui explique en partie pourquoi celle-ci fonctionne.

²⁰ Notez que j'utilise ici la notation de manière abusive : auparavant, les exposants désignaient les couches dans le transformateur, mais ici, je les utilise pour désigner le nombre d'éléments dans la séquence d'entrée.

alors 1. au moment du test, la génération autorégressive peut utiliser des mises à jour incrémentielles pour calculer efficacement la nouvelle représentation, 2. au moment de l'entraînement, nous pouvons faire les N prédictions autorégressives pour toute la séquence $p(w_1 = w)p(w_2 = w|w_1)p(w_3 = w|w_1, w_2) \dots p(w_N = w|w_{1:N-1})$ en un seul passage vers l'avant.

Malheureusement, le transformateur standard présenté ci-dessus ne possède pas cette propriété en raison de la forme de l'attention utilisée. Chaque token prête attention à tous les autres tokens, donc si nous ajoutons un nouveau token à la séquence, la représentation de chaque token change dans tout le transformateur. Cependant, si nous masquons la matrice d'attention de manière à ce qu'elle soit triangulaire supérieure $A_{n,n} = 0$ lorsque $n > n'$, alors la représentation de chaque mot *ne dépend que des mots précédents*.²¹ Cela nous donne alors la propriété incrémentielle, car aucune des autres opérations du transformateur n'opère sur l'ensemble de la séquence.²²

Ajout d'une tête. Nous sommes maintenant presque prêts à effectuer la modélisation autorégressive du langage. Nous appliquons le bloc de transformateur masqué M fois à la séquence d'entrée de mots. Nous prenons ensuite la représentation au token $n - 1$, c'est-à-dire $x^{(M)}$ qui capture les informations causales dans la séquence à ce stade, et nous générons la probabilité du mot suivant grâce à une opération softmax

$$p(w_n = w|w_{1:n-1}) = p(w_n = w|x^{(M)}) = \frac{\exp(\mathbf{g}_w^T \mathbf{x}_{n-1}^{(M)})}{\sum_{w=1}^W \exp(\mathbf{g}_w^T \mathbf{x}_{n-1}^{(M)})}.$$

Ici, W est la taille du vocabulaire, le mot w est $w \in \{g_w\}^W$ et $\mathbf{g}_w$ sont des fonctions softmax sont les poids qui seront appris.

4.2 Classification d'images

Pour la classification d'images, l'objectif est de prédire l'étiquette y à partir de l'image d'entrée qui a été tokenisée en séquence $X^{(0)}$, c'est-à-dire $p(y|X^{(0)})$. Une façon de calculer cette distribution serait d'appliquer le corps du transformateur standard M fois aux fragments d'image tokenisés avant d'agréger la couche finale du

transformateur, $X^{(M)}$, à travers la séquence, par exemple par regroupement spatial $\mathbf{h} = \sum_{n=1}^{N_n} \mathbf{x}_n^{(M)}$ afin de former une représentation des caractéristiques pour l'ensemble de l'image. La représentation $\mathbf{h}$ pourrait alors être utilisée pour effectuer une classification softmax. Une autre approche s'avère plus performante [Dosovitskiy et al., 2021]. Nous introduisons plutôt un nouveau jeton fixe (appris) au début $n = 0$ de la séquence d'entrée $x^{(0)}$. À la tête, nous utilisons le $x_0 = 0$, pour effectuer la classification softmax.

Cette approche présente l'avantage que le transformateur maintient et affine une représentation globale de la séquence à chaque couche m du transformateur qui est appropriée pour la classification.

4.3 Utilisations plus complexes

Le bloc transformateur peut également être utilisé dans le cadre de systèmes plus complexes. Par exemple, dans les architectures encodeur-décodeur pour la modélisation séquence-à-séquence pour la traduction [Devlin et al., 2019, Vaswani et al., 2017] ou dans les auto-encodeurs masqués pour les systèmes de vision auto-supervisés [He et al., 2021].

5 Conclusion

Ceci conclut cette introduction de base aux transformateurs, qui se voulait mathématiquement précise et visait à fournir des intuitions derrière les décisions de conception.

Nous n'avons pas abordé en détail les fonctions de perte ou l'entraînement, mais cela s'explique par le fait que des approches d'apprentissage profond plutôt standard sont utilisées à cet effet. En bref,

²¹ Notez que cette opération de masquage encode également les informations de position, car vous pouvez déduire l'ordre des tokens à partir du masque.

²² Cette restriction de l'attention entraînera une perte de pouvoir de représentation. La question reste ouverte quant à l'importance de ce phénomène et à la possibilité de l'atténuer en augmentant la capacité du modèle, par exemple en utilisant des jetons de dimension supérieure, c'est-à-dire en augmentant D .

Les transformateurs sont généralement entraînés à l'aide de l'optimiseur Adam. Leur entraînement est souvent lent par rapport à d'autres architectures et ils deviennent généralement plus instables à mesure que l'entraînement progresse. Le clippage des gradients, les programmes de taux d'apprentissage décroissants et l'augmentation de la taille des lots au cours de l'entraînement contribuent à atténuer ces instabilités, mais celles-ci persistent souvent.

Remerciements. Nous remercions le Dr Max Patacchiola, Sasha Shysheya, John Bronskill, Runa Eschenhagen et Jess Riedel pour leurs commentaires sur les versions précédentes de cette note. Richard E. Turner bénéficie du soutien de Microsoft, Google, Amazon, ARM, Improbable et de la subvention EP/T005386/1 de l'EPSRC.

Références

Jimmy Lei Ba, Jamie Ryan Kiros et Geoffrey E Hinton. Normalisation des couches.
Prépublication arXiv arXiv:1607.06450, 2016.

Kaifeng Bi, Lingxi Xie, Hengheng Zhang, Xin Chen, Xiaotao Gu et Qi Tian. Pangu-weather : un modèle 3D haute résolution pour des prévisions météorologiques mondiales rapides et précises, 2022.

Jacob Devlin, Ming-Wei Chang, Kenton Lee et Kristina Toutanova. BERT : pré-entraînement de transformateurs bidirectionnels profonds pour la compréhension du langage. Dans *les actes de la conférence 2019 de la section nord-américaine de l'Association for Computational Linguistics : Human Language Technologies, volume 1 (articles longs et courts)*, pages 4171-4186, Minneapolis, Minnesota, juin 2019. Association for Computational Linguistics. doi : 10.18653/v1/N19-1423. URL <https://aclanthology.org/N19-1423>.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xi-aohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit et Neil Houlsby. Une image vaut 16x16 mots : Transformers pour la reconnaissance d'images à grande échelle. Dans *le cadre de la 9e Conférence internationale sur l'apprentissage des représentations, ICLR 2021, événement virtuel, Autriche, 3-7 mai 2021*. OpenReview.net, 2021. URL <https://openreview.net/forum?id=YicbFdNTTy>.

Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár et Ross Girshick. Les auto-encodeurs masqués sont des outils d'apprentissage visuel évolutifs, 2021.

Mary Phuong et Marcus Hutter. Algorithmes formels pour les transformateurs.
Prépublication arXiv arXiv:2207.09238, 2022.

Sheng Shen, Zhewei Yao, Amir Gholami, Michael Mahoney et Kurt Keutzer. PowerNorm : repenser la normalisation par lots dans les transformateurs. Dans Hal Daumé III et Aarti Singh, éditeurs, *Actes de la 37e Conférence internationale sur l'apprentissage automatique*, volume 119 des *Actes de la recherche sur l'apprentissage automatique*, pages 8741-8751. PMLR, 13-18 juillet 2020. URL <https://proceedings.mlr.press/v119/shen20e.html>.

Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke et Alexander Alemi. Inception-v4, inception-resnet et l'impact des connexions résiduelles sur l'apprentissage. Dans *les actes de la conférence AAAI sur l'intelligence artificielle*, volume 31, 2017.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser et Illia Polosukhin. L'attention est tout ce dont vous avez besoin. Dans I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach,

R. Fergus, S. Vishwanathan et R. Garnett, éditeurs, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

K. Wu, H. Peng, M. Chen, J. Fu et H. Chao. Repenser et améliorer le codage de position relative pour le transformateur de vision. Dans *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 10013–10021, Los Alamitos, Californie, États-Unis, octobre 2021. IEEE Computer Society. doi : 10.1109/ICCV48922.2021.00988. URL <https://doi.ieeecomputersociety.org/10.1109/ICCV48922.2021.00988>.

Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang et Tieyan Liu. À propos de la normalisation des couches dans l'architecture du transformateur. Dans Hal Daumé III et Aarti Singh, éditeurs, *Actes de la 37e Conférence internationale sur l'apprentissage automatique*, volume 119 des *Actes de la recherche sur l'apprentissage automatique*, pages 10524-10533. PMLR, 13-18 juillet 2020. URL <https://proceedings.mlr.press/v119/xiong20b.html>.